

Build Your Own Search Engine

Complete 14-Week Hardcore CS Fundamentals Plan

C++ · OS · Networking · IR · DBMS · NLP · Docker · Testing · Evaluation · Observability

Enhanced Edition v3 — Adds: Week 11 (Deep Testing), Week 12 (Observability & Load Testing), Week 13 (Advanced IR & Learning-to-Rank), Week 14 (Production Hardening & Capstone) Plus: complete testing frameworks, latency/throughput benchmarks, CI/CD pipelines.

■ Concepts

■ Resources

◆ Build Tasks

★ New/Extended

■ Critical Note

Table of Contents

PHASE 0 (Day 0)	· Setup + Mental Model + Architecture
WEEK 1	· C++ Core — Pointers, Memory, STL (Days 1–7)
WEEK 2	· OS + Threading + Concurrency (Days 8–14)
WEEK 3	· Networking — TCP/IP, Sockets, HTTP (Days 15–21)
WEEK 4	· Web Crawler — Full Implementation (Days 22–28)
WEEK 5	· NLP Basics — Search Pipeline (Days 29–35)
WEEK 6	· Database Systems (Days 36–42)
WEEK 7	· Information Retrieval — Core (Days 43–49)
WEEK 8	· Backend — FastAPI (Days 50–52)
WEEK 9	· Frontend — Jinja2 + Search UI (Days 53–56)
WEEK 10	· Deployment — Docker + Nginx (Days 57–60)
★ WEEK 11	· Deep Testing — Unit, Integration, Stress (Days 61–67)
★ WEEK 12	· Observability, Load Testing & Profiling (Days 68–74)
★ WEEK 13	· Advanced IR — LTR, Vector Search, Neural (Days 75–81)
★ WEEK 14	· Production Hardening + Capstone Review (Days 82–90)
Appendix A	· System Architecture Quick Reference
Appendix B	· Complete Testing & Benchmarking Reference
Appendix C	· Indian Language NLP Reference
Appendix D	· Evaluation Metrics Cheat Sheet
Appendix E	· Performance Targets & SLA Reference

What Was Missing from the v2 Plan — Gap Analysis

The v2 plan (10 weeks) is an excellent foundation but leaves several production-critical areas either absent or underspecified. This v3 plan adds 4 full weeks and expands every appendix to fill these gaps:

Gap Area	Problem in v2	Fixed in v3
Unit & Integration Testing	No test framework setup, no coverage targets, no mocking strategy	Week 11: GoogleTest (C++), pytest, test doubles, coverage gates
Stress / Load Testing	Locust mentioned once in Week 8 with no framework or targets	Week 12: Locust scenarios, wrk2, k6, throughput/latency targets
Profiling & Flame Graphs	Not mentioned at all	Week 12: perf, gprof, Valgrind callgrind, py-spy, flamegraphs
Observability (Metrics/Logs/Traces)	Prometheus+Grafana mentioned briefly in Week 10	Week 12: full instrumentation, structured logs, Jaeger tracing
CI/CD Pipeline	Not mentioned at all	Week 14: GitHub Actions, Docker build, test gates, deploy pipeline
Learning-to-Rank (LTR)	Listed as 'Advanced Topic' with 1 line	Week 13: click log collection, feature engineering, LambdaMART
Vector / Semantic Search	1-line mention of FAISS	Week 13: embeddings, HNSW, ANN benchmarking, hybrid BM25+dense
Fault Injection & Chaos	Not mentioned	Week 14: kill containers, inject latency, test retry/backoff logic
Performance SLA Targets	Only '<100ms query latency' mentioned	Appendix E: full SLA table for every component
Benchmark Baselines	No reference baselines given	Appendix B: baselines for indexer throughput, crawler speed, etc.
Security Hardening	Not mentioned	Week 14: input sanitisation, rate limiting, robots.txt edge cases
Capstone Review Checklist	Chapter 11 exists but not rigorous	Week 14: full rubric with pass/fail criteria for every milestone

PHASE 0 (Day 0) — Setup + Mental Model

Day 0 — Environment + Blueprint + Testing Mindset

Orient yourself before writing a single line of code. Install the full toolchain, read the blueprint, sketch the data-flow diagram, and set up your testing infrastructure from Day 0.

Core Concepts:

- Install: gcc/g++ (≥C++17), cmake, docker, postgres, redis, valgrind, wireshark/tcpdump, python3, pip
- Testing tools from Day 0: googletest, pytest, locust, wrk2, hyperfine — install now
- Observability tools: prometheus-client, grafana (docker), jaeger (docker)
- Sketch: crawler → raw HTML → NLP pipeline → inverted index → query engine → API → UI
- Full data-flow: Seed URLs → Redis Frontier → C++ Crawler → MinIO → Kafka → Python NLP → RocksDB → QueryEngine → FastAPI → Nginx
- CAP positions: Redis URL frontier = CP; Postgres + RocksDB = AP (accept stale, re-crawl on 404)

Build Tasks:

- ◆ Install all tools and verify with `--version` / `docker ps`
- ◆ Read blueprint PDF end-to-end
- ◆ Sketch data-flow on paper/whiteboard
- ◆ Create GitHub repo with folder structure from blueprint
- ◆ Set up pre-commit hooks: `clang-format` for C++, `black` for Python
- ◆ Write one-paragraph description of each microservice in your own words
- ◆ ★ Create a TESTING.md in your repo listing what you will test for each service and what 'done' means
- Do NOT skip Day 0. Architecture clarity and testing discipline now saves weeks of debugging later.

WEEK 1 — C++ Core: Pointers, Memory, STL

Days 1–7. Master C++ memory model before any system-level code. Every concept here appears directly in the crawler.

Day 1 — Pointers & References

Core Concepts:

- Raw vs smart pointers
- Pointer arithmetic
- Pass-by-pointer vs pass-by-reference
- nullptr
- void* and casting
- Function pointers for thread pool callbacks

Build Tasks:

- ◆ Reverse array with pointer arithmetic only
 - ◆ Build dynamic array with new/delete
 - ◆ swap() with pointers vs references — compare assembly
 - ◆ Callback system with function pointers
-

Day 2 — Stack vs Heap + Memory Management

Core Concepts:

- Stack frame lifecycle
- Heap allocation new/delete
- Memory leaks & detection
- Buffer overflows
- Fragmentation & slab allocators
- AddressSanitizer (ASan)

Build Tasks:

- ◆ Simulate memory leak → detect with valgrind --leak-check=full → fix
 - ◆ Stack-based string class → observe stack overflow
 - ◆ Compile -fsanitize=address → reproduce buffer overflow
-

Day 3 — RAII + Smart Pointers

Core Concepts:

- RAII principle
- unique_ptr — zero overhead exclusive ownership
- shared_ptr — ref counting
- weak_ptr — break cycles
- Move semantics std::move
- make_unique / make_shared

Build Tasks:

- ◆ Rewrite Day 1 array using unique_ptr
 - ◆ shared_ptr reference count graph
 - ◆ Dangling pointer bug → fix with smart pointer
 - ◆ RAII file-handle wrapper (open on construct, close on destruct)
-

Day 4 — STL Deep Dive

Core Concepts:

- vector — amortised $O(1)$ push_back
- unordered_map — hash table
- map — red-black tree $O(\log n)$
- queue/deque for BFS frontiers
- Iterators and range-based for
- priority_queue for URL scheduling

Build Tasks:

- ◆ Word frequency counter on 1MB text with unordered_map
 - ◆ Priority queue max-heap for URL scheduling by score
 - ◆ Benchmark vector.push_back vs reserve+push_back
 - ◆ Simulate URL frontier: priority_queue>
-

Day 5 — Time Complexity + Cache Locality

Core Concepts:

- Big-O: average vs worst vs amortised
- CPU cache hierarchy: L1~4cy, L2~12cy, L3~40cy, RAM~200cy
- Cache line = 64 bytes, spatial locality
- False sharing in multithreading
- Data-Oriented Design: SoA vs AoS
- Posting list iteration is cache-critical

Build Tasks:

- ◆ Benchmark vector vs list iteration — ns/element with std::chrono
 - ◆ SoA vs AoS on 1M elements
 - ◆ False sharing demo: two threads on adjacent vs separate cache lines
 - ◆ Sequential sorted-array scan vs unordered_map random access — quantify cache miss penalty
-

Day 6 — File I/O + Serialisation

Core Concepts:

- fstream: ifstream, ofstream — open modes
- Buffered I/O, setvbuf
- Binary vs text mode, endianness
- mmap concept — zero-copy file access
- Content-addressed storage: SHA256(body) as filename

Build Tasks:

- ◆ Write crawled HTML with SHA256 content-addressed filenames
 - ◆ Read back, parse first 100 bytes, measure throughput MB/s
 - ◆ mmap-based reader vs fstream — speed comparison
 - ◆ Simulate MinIO store: dict[sha256]→filepath — duplicate HTML stores once
-

Day 7 — Mini Project: LRU Cache

Core Concepts:

- LRU eviction: HashMap + DLL for $O(1)$ get/put
- Cache hit/miss ratio
- Thread-safe LRU with mutex
- TTL-aware LRU for DNS entries

Build Tasks:

- ◆ LRU Cache from scratch — no hints, no library
- ◆ 1M random get/put — measure throughput

- ◆ Add mutex — re-benchmark
- ◆ TTL-aware LRU: entries expire after N seconds
- ◆ Wire into stub DNS resolver — 300s TTL — observe cache hit
- Week 1 Output: LRU Cache + memory-safe dynamic array with smart pointers + unit tests for both.

WEEK 2 — OS + Threading + Concurrency

Days 8–14. Your crawler is multi-threaded. Understand what the OS gives each thread and what is shared.

Day 8 — Processes vs Threads

Core Concepts:

- Process: private address space, PCB, fd table
- Thread: shared heap/globals/fds, private stack/registers
- Context switching: register save/restore, TLB flush
- User threads vs kernel threads — 1:1 vs M:N
- `fork()` vs thread creation — `fork` copies page tables (CoW)
- Thread-local storage (TLS) — per-thread HTTP fetcher avoids locking

Build Tasks:

- ◆ `fork()` 10 times, measure latency with `clock_gettime()`
 - ◆ Compare with thread creation — expect 10-100x difference
 - ◆ `/proc/{pid}/maps` to visualise address space
 - ◆ `thread_local` DNSResolver — 8 threads, each resolving different hostname — no locking
-

Day 9 — Threading in C++

Core Concepts:

- `std::thread`: creation, `join()`, `detach()`, lifecycle
- Lambda captures — value vs reference (dangling refs!)
- `sleep_for`, `get_id()`, `hardware_concurrency()`
- RAII thread wrapper — join-on-destruction
- `std::jthread` (C++20) — auto-join + `stop_token`

Build Tasks:

- ◆ 10 threads fetching URL segments — benchmark sequential vs parallel
 - ◆ Deliberately leak a joinable thread — observe `std::terminate`
 - ◆ Re-implement with `std::jthread` + `stop_token` — clean cancellation
-

Day 10 — Mutex + Locks + Race Conditions

Core Concepts:

- Race condition: shared counter classic example
- `std::mutex`: `lock()/unlock()`
- `std::lock_guard` — RAII scoped lock
- `std::unique_lock` — deferred, timed, conditional
- Deadlock: circular wait — prevention via lock ordering
- `std::shared_mutex` — multiple readers / single writer for DNS cache

Build Tasks:

- ◆ Reproduce race condition — observe non-deterministic output
 - ◆ Fix with `std::mutex` — confirm deterministic output
 - ◆ Deliberately deadlock (two threads, two mutexes, opposite order)
 - ◆ Fix with lock ordering
 - ◆ DNS LRU with `shared_mutex` — readers concurrent, writer upgrades on miss
 - Your URL queue will use mutex. Get race conditions and deadlocks deeply right here.
-

Day 11 — Condition Variables + Semaphores

Core Concepts:

- Spurious wakeups — always while loop, never if
- Producer-consumer: bounded buffer problem
- `std::condition_variable`: `wait()`, `notify_one()`, `notify_all()`
- Counting semaphore (C++20)
- Back-pressure: if Kafka full, frontier blocks

Build Tasks:

- ◆ **Producer-consumer queue with `condition_variable`**
 - ◆ **4 producers + 2 consumers — print events**
 - ◆ **Spurious wakeup bug (if) vs fix (while)**
 - ◆ **Bounded capacity 1000 items — producer blocks when full (Kafka back-pressure sim)**
-

Days 12–13 — Thread Pool

Core Concepts:

- Fixed-size pool: N workers running forever
- Task queue: `std::function` + `std::queue` + mutex
- Worker loop: wait on cv → dequeue → execute
- Graceful shutdown: stop flag + `notify_all()`
- Work stealing: per-thread dequeues

Build Tasks:

- ◆ **Thread pool from scratch — no library**
 - ◆ **1000 simulated fetch tasks — throughput vs single-thread**
 - ◆ **Graceful shutdown — join all workers**
 - ◆ **future/promise return from enqueue so callers can wait on results**
 - ◆ **Scale to 64 threads — 10K simulated URL fetches — tasks/sec**
-

Day 14 — Week 2 Testing Checkpoint

Core Concepts:

- Thread sanitiser (TSan) — detects data races at runtime
- Helgrind (Valgrind) — deadlock and lock-order analysis
- Stress tests: run thread pool under heavy load and kill it

Build Tasks:

- ◆ **Compile all Week 2 code with `-fsanitize=thread` — zero races**
- ◆ **Run Helgrind on thread pool — zero deadlock warnings**
- ◆ **Stress test: 10K tasks submitted from 8 producer threads simultaneously — no crash, no dropped tasks**
- ◆ **Measure thread pool throughput: tasks/sec vs thread count (1,2,4,8,16,32,64) — plot scaling**
- **Week 2 Output: Production thread pool + zero TSan/Helgrind warnings + throughput benchmark chart.**

WEEK 3 — Networking: TCP/IP, Sockets, HTTP

Days 15–21. Every HTTP request your crawler makes travels through these layers.

Days 15–16 — TCP/IP Model

Core Concepts:

- OSI vs TCP/IP layers
- IP: addressing, routing, TTL, fragmentation
- TCP: 3-way handshake, sequence numbers, sliding window
- UDP vs TCP tradeoffs
- Ports, sockets, ephemeral ports, NAT
- TIME_WAIT state — SO_REUSEADDR fixes crawler socket lingering

Build Tasks:

- ◆ Trace HTTP request with tcpdump — save pcap
 - ◆ Open in Wireshark — identify SYN, SYN-ACK, ACK, GET, 200 OK
 - ◆ SO_REUSEADDR on server socket — restart immediately after kill
-

Days 17–18 — Socket Programming

Core Concepts:

- socket()→bind()→listen()→accept() server flow
- connect() client — TCP handshake triggered here
- send()/recv() and partial reads — recv returns up to N bytes
- Non-blocking sockets: O_NONBLOCK + select/poll/epoll
- Socket options: SO_REUSEADDR, TCP_NODELAY, SO_TIMEOUT
- Everything is a file descriptor — UNIX philosophy

Build Tasks:

- ◆ TCP echo server (receive, send back)
 - ◆ TCP client — raw HTTP GET — parse response manually
 - ◆ Handle partial recv() correctly — loop until full response
 - ◆ Non-blocking connect() with select() timeout
 - ◆ Replicate blueprint HTTPFetcher::fetch() step-by-step
 - Read Beej's Guide chapters 1–7 cover to cover. This week decides your networking career.
-

Day 19 — DNS Resolution

Core Concepts:

- DNS hierarchy: root→TLD→authoritative→resolver
- Record types: A, AAAA, CNAME, MX
- TTL and DNS caching — LRU cache from Day 7 applies
- getaddrinfo() API — thread-safe
- /etc/hosts and /etc/resolv.conf — local overrides

Build Tasks:

- ◆ DNS resolver using getaddrinfo() in C++
 - ◆ Cache resolved IPs with TTL expiry
 - ◆ Measure latency before and after cache hit
 - ◆ Implement blueprint DNSResolver — 20 hostnames — confirm cache hit rate
-

Day 20 — HTTP Deep Dive

Core Concepts:

- HTTP/1.1 request/response: start-line, headers, blank line, body
- Key headers: Host, Content-Type, User-Agent, Accept-Encoding
- Status codes: 200, 301/302, 404, 429, 503
- Keep-alive — reuse TCP connections
- Chunked transfer encoding
- HTTP/2 framing concept — multiplexing, HPACK

Build Tasks:

- ◆ Parse raw HTTP response manually — extract status, headers, body
 - ◆ Handle 301 redirect chains up to depth 5
 - ◆ Implement basic gzip decompression (zlib/inflate)
 - ◆ Reproduce blueprint request string exactly with IndiaBot/1.0 User-Agent
-

Day 21 — Mini Project: HTTP Client in C++

Core Concepts:

- Connection pooling — reuse sockets across requests to same host
- Timeout handling with select()
- Error handling: ECONNREFUSED, ETIMEDOUT
- SSL/TLS concept — OpenSSL SNI for HTTPS

Build Tasks:

- ◆ Fetch https://example.com — print status + first 500 bytes
- ◆ Redirects up to depth 5 automatically
- ◆ 100 sequential requests to same host — measure keep-alive vs fresh
- ◆ Add OpenSSL TLS wrapping — HTTPS end-to-end
- Week 3 Output: C++ HTTP client with DNS, redirect handling, connection reuse + integration test against real servers.

WEEK 4 — Crawler: Full Implementation

Days 22–28. Combine thread pool + URL queue + HTTP client into a scalable crawler.

Day 22 — URL Parsing + Normalisation

Core Concepts:

- RFC 3986: scheme, authority, path, query, fragment
- Percent-encoding — decode before comparison
- Relative URL resolution
- Normalisation: lowercase, remove fragment, sort query params
- Canonicalisation: strip utm_source, fbclid
- Per-domain crawl budget

Build Tasks:

- ◆ Parse 20 malformed URLs correctly
 - ◆ Resolve relative URLs — handle .././../ paths
 - ◆ Strip tracking params from 1000 URLs
 - ◆ Domain extractor — group 10K URLs by domain — enforce 500 pages/domain budget
-

Day 23 — robots.txt + Politeness

Core Concepts:

- robots.txt syntax: User-agent, Disallow, Allow, Crawl-delay
- Wildcard matching: * and \$ in paths
- Per-host rate limiting with token bucket
- 429 → exponential backoff with jitter
- Sitemap.xml discovery

Build Tasks:

- ◆ Parse robots.txt for 5 real sites — print allowed/disallowed
 - ◆ Per-host rate limiter with token bucket
 - ◆ Exponential backoff on 429 — test up to 5 retries
 - ◆ Wire politeness: per-domain timestamp map (shared_mutex) — block until delay elapsed
-

Days 24–25 — Multithreaded Crawler Architecture

Core Concepts:

- Frontier queue: FIFO vs priority
- Per-host queues — isolate for politeness
- URL scheduling: BFS vs DFS vs best-first
- Back-pressure — don't enqueue faster than you can crawl
- Crawl depth limits — prevent cycles

Build Tasks:

- ◆ Crawler fetches 100 URLs with 8 threads — per-host rate limiting active
 - ◆ Saves HTML + metadata JSON to disk
 - ◆ Replace disk queue with Redis BLPOP frontier
 - ◆ Send fetched HTML to mock Kafka topic
-

Day 26 — Bloom Filter for URL Deduplication

Core Concepts:

- Bloom filter: bit array + k hash functions, O(k) insert/query

- False positive rate formula: $(1 - e^{(-kn/m)})^k$
- k and m for 0.1% FPR at 1B URLs $\rightarrow \sim 2\text{GB}$
- Counting Bloom filter — supports deletions
- Scalable Bloom filter — chains for unbounded growth

Build Tasks:

- ◆ Implement Bloom filter: insert + query from scratch
 - ◆ Measure FPR empirically at 100K, 500K, 1M inserts
 - ◆ Compare memory: Bloom vs unordered_set at 1M URLs
 - ◆ Use RedisBloom BF.ADD/BF.EXISTS — 1M URLs — confirm distributed dedup
-

Days 27–28 — Full Crawler Integration

Core Concepts:

- Checkpoint + resume — save frontier every 1000 pages
- Crawl stats: pages/sec, errors, redirects, bytes fetched
- Duplicate content detection — SHA256 of response body
- Link extraction — lxml/BeautifulSoup
- Per-domain crawl budget

Build Tasks:

- ◆ Crawl 500+ pages from seed URLs
- ◆ Checkpoint and resume after intentional interrupt
- ◆ Output: per-page JSON with url, title, links[], timestamp, content_hash
- ◆ Publish to Kafka 'raw_pages' — verify consumer receives all
- ◆ 4 crawler replicas via docker-compose scale — no URL duplication via Bloom
- Week 4 Output: Multithreaded C++ crawler: 500+ pages, dedup, rate-limiting, checkpointing, Kafka integration.

WEEK 5 — NLP Basics: Search Pipeline

Days 29–35. Build the complete document processing pipeline: raw HTML in → structured JSON out.

Day 29 — Text Preprocessing

Core Concepts:

- HTML tag stripping vs structured parsing
- Unicode normalisation: NFC vs NFD
- Whitespace normalisation
- Boilerplate removal: nav, footer, ads
- Character encoding detection — chardet

Build Tasks:

- ◆ Strip HTML from 100 crawled pages — measure text yield
 - ◆ Normalise whitespace and Unicode (NFC) across all pages
 - ◆ Remove boilerplate with text-density heuristic (>50 words per block)
 - ◆ Implement blueprint `extract_text()` — title, description, body, h1, h2
-

Day 30 — Tokenisation + Stemming + Lemmatisation

Core Concepts:

- Word tokenisation: whitespace vs regex vs BPE
- Sentence tokenisation — Punkt
- Stemming (Porter, Snowball): fast, lossy
- Lemmatisation (WordNet): accurate, slow
- Stop word lists — when NOT to remove

Build Tasks:

- ◆ Tokenise + stem 10,000 documents with NLTK Snowball
 - ◆ Compare Porter vs Snowball on 100 sample words
 - ◆ Build bigram index alongside unigram — measure size difference
 - ◆ Blueprint `tokenize()`: English→Snowball, Indian langs→Stanza, fallback→`indic_tokenize`
-

Day 31 — TF-IDF

Core Concepts:

- TF: raw vs log-normalised vs augmented
- IDF: $\log(N/df)$ — rare terms rewarded
- Smooth IDF: $\log(N+1/df+1)+1$
- TF-IDF vector space model
- Cosine similarity — magnitude-normalised dot product

Build Tasks:

- ◆ TF-IDF for 1000 docs with scikit-learn
 - ◆ Top-5 similar docs to query with cosine similarity
 - ◆ TF-IDF from scratch (no sklearn) — compare outputs
 - ◆ Apply blueprint field weights: $3.0*TF_title + 1.0*TF_body$ — compare ranking
-

Days 32–35 — Full NLP Pipeline + Language Detection

Core Concepts:

- FastText language ID — 176 languages, very fast
- Content quality signals: text density, link density

- Near-duplicate detection: shingling + MinHash
- NER: spaCy for English, Stanza for Indian langs
- Keyword extraction — YAKE, RAKE — unsupervised

Build Tasks:

- ◆ Process 1000 HTML pages through full pipeline
- ◆ Output: {url, title, tokens, lang, keywords, entities, timestamp}
- ◆ Filter: remove pages <50 words
- ◆ Kafka consumer reading 'raw_pages' → pipeline → write to 'processed_pages'
- ◆ index_emitter.py: emit (term, lang, doc_id, tf, positions[]) to indexer
- Week 5 Output: Python NLP pipeline: HTML → structured JSON + language detection + Kafka integration.

WEEK 6 — Database Systems

Days 36–42. Understand how your storage layer actually works under the hood.

Days 36–37 — PostgreSQL + Indexing

Core Concepts:

- B-tree index: range queries, uniqueness, composite ordering
- Hash index: equality-only
- GIN index: inverted index for FTS, JSONB, arrays
- EXPLAIN ANALYZE: Seq Scan vs Index Scan vs Index Only Scan
- VACUUM, autovacuum, dead tuples — UPDATE/DELETE bloat

Build Tasks:

- ♦ Schema: `documents(id, url, title, lang, crawled_at, word_count)` + indexes
 - ♦ EXPLAIN ANALYZE query filtering by lang AND date — before/after index
 - ♦ Implement full blueprint schema: `urls`, `links`, `entities`, `query_log`
 - ♦ JOIN query: top-10 English URLs by PageRank in last 7 days — verify Index Scan
-

Days 38–39 — Redis + RocksDB

Core Concepts:

- Redis: String, List, Sorted Set, Hash data types
- LPUSH/RPOP (FIFO), BLPOP (blocking pop)
- RocksDB LSM-tree: MemTable→SSTable→Compaction
- Write-ahead log (WAL) for durability
- LSM vs B-tree: LSM better for writes, B-tree for point reads

Build Tasks:

- ♦ URL frontier in Redis List (LPUSH seeds, BLPOP workers)
 - ♦ DNS cache with TTL using SET EX
 - ♦ 100K term→postings in RocksDB — benchmark read latency
 - ♦ `encode_postings()` with delta encoding + `struct.pack` + `zstd level-3` — measure compression ratio
-

Days 40–42 — Data Layer Integration

Core Concepts:

- Metadata → PostgreSQL (relational, ACID)
- Posting lists → RocksDB (write-optimized)
- URL queue + cache → Redis (in-memory, TTL)
- Connection pooling — pgBouncer concept
- Write-through vs write-behind caching

Build Tasks:

- ♦ 10K crawled documents across all three systems
- ♦ Query: English docs crawled today, `word_count > 500`
- ♦ Measure write throughput for batch inserts into each system
- ♦ ClickHouse as analytics store — `query_log` events — CTR-by-rank aggregation
- Week 6 Output: 3-tier storage — Postgres + Redis + RocksDB — with benchmark numbers for each.

WEEK 7 — Information Retrieval: Core

Days 43–49. The search engine's brain. Every search engine is built on these algorithms.

Day 43 — Inverted Index

Core Concepts:

- Inverted index: term $\rightarrow$ postings list (docID, tf, positions[])
- Dictionary: hash table or B-tree
- Postings list sorted by docID for merge efficiency
- Positional index for phrase queries
- Skip pointers for long lists
- Compression: delta encoding, variable byte encoding

Build Tasks:

- ◆ Inverted index for 10K documents in Python
 - ◆ AND/OR queries via postings intersection/union
 - ◆ Measure: index size vs raw text size, query latency
 - ◆ Field weights during indexing: title=3.0, body=1.0
-

Days 44–45 — Index Builder + Compression

Core Concepts:

- SPIMI: single-pass in-memory indexing
- External merge sort — merge partial indexes without loading all into RAM
- Variable byte encoding: 1 byte per small int
- Delta encoding for doc IDs
- Index partitioning: term-based vs doc-based

Build Tasks:

- ◆ 50K documents with SPIMI — flush every 10K docs
 - ◆ Compress postings with delta + VByte — measure compression ratio
 - ◆ Store merged index in RocksDB
 - ◆ `get_shard()` (MD5 mod 8) — terms distribute evenly across 8 instances
 - ◆ Query fan-out: broadcast to 8 shards, merge top-k — measure latency
-

Day 46 — BM25 Ranking

Core Concepts:

- BM25: TF saturation k_1 , length normalisation b
- k_1 (1.2–2.0): how much repeated terms matter
- b (0.75): length normalisation degree
- IDF smoothing: $\log((N-df+0.5)/(df+0.5)+1)$
- BM25+: lower bound delta fix
- BM25F: different k_1/b per field

Build Tasks:

- ◆ BM25 from scratch (no library) in Python
 - ◆ Tune k_1 and b — find best $nDCG@10$ on sample query set
 - ◆ Compare BM25 vs TF-IDF recall@10
 - ◆ Wire blueprint BM25 class to RocksDB posting list reader
-

Day 47 — PageRank

Core Concepts:

- Random surfer model
- $PR(u) = (1-d)/N + d \cdot \sum (PR(v)/L(v))$
- Damping factor $d=0.85$
- Power iteration until convergence (~50 iterations)
- Dangling nodes — redistribute rank
- TrustRank — seed trusted pages

Build Tasks:

- ◆ PageRank on crawled link graph
 - ◆ Top-10 most authoritative pages
 - ◆ Experiment with damping factors 0.7, 0.85, 0.95
 - ◆ blueprint `compute_pagerank()`: 50 iterations, damping=0.85, UPDATE urls SET pagerank
-

Days 48–49 — Query Engine + Result Ranking

Core Concepts:

- Query pipeline: tokenise→stem→lookup→score→rank
- DAAT vs TAAT query processing
- Top-k retrieval with min-heap
- Score combination: $BM25 \cdot 0.7 + PageRank \cdot 0.3$
- Snippet generation: best 40-word window
- Spell correction: edit distance + LM prior

Build Tasks:

- ◆ Query engine top-10 results in <100ms for 10K doc index
- ◆ Highlighted snippets for each result
- ◆ BM25+PageRank vs BM25-only ranking comparison
- ◆ `generate_snippet()`: best 40-word window
- ◆ Cross-lingual retrieval: Roman Hindi → transliterate → Devanagari → merge results
- Week 7 Output: Search engine: inverted index + BM25 + PageRank + <100ms query latency.

WEEKS 8–9 — Backend (FastAPI) + Frontend

Days 50–56. Expose search features through a clean REST API and build the search UI.

Days 50–52 — FastAPI Search API

Core Concepts:

- Async/await — event loop, coroutines, asyncio
- Path params, query params, Pydantic validation
- GET /search?q=&page;= — paginated with Redis cache
- Rate limiting with slowapi — 100 req/min per IP
- OpenAPI auto-documentation — /docs

Build Tasks:

- ◆ /search endpoint with pagination (10 results/page)
 - ◆ Cache repeated queries in Redis for 60s
 - ◆ Load test with Locust — sustain 100 req/s — profile bottlenecks
 - ◆ /autocomplete via PostgreSQL trigram index (pg_trgm)
 - ◆ /api/click: receives {session_id, query, clicked_pos, doc_id} → writes to query_log
-

Days 53–56 — Search UI — Jinja2 + HTML/CSS

Core Concepts:

- Jinja2 template inheritance — base.html + blocks
- Result card: rank, title, URL, snippet, score, entity tags
- Term highlighting in snippets
- Pagination with prev/next
- HTMX for search-as-you-type without React

Build Tasks:

- ◆ Search results page with snippets + query term highlighting
- ◆ Pagination 10 results/page with prev/next
- ◆ Mobile-responsive layout using CSS flexbox/grid
- ◆ Click tracking: log (session_id, query, clicked_rank) to Postgres
- ◆ Language selector (auto/en/hi/ta), entity tag badges, trackClick() beacon
- ◆ Dwell time: timestamp on click, send elapsed on page unload to /api/dwell
- Week 8–9 Output: FastAPI API + Jinja2 UI with pagination, Redis caching, click/dwell tracking.

WEEK 10 — Deployment: Docker + Nginx

Days 57–60. Package every service into Docker containers and deploy the full stack.

Days 57–60 — Full Stack Deployment

Core Concepts:

- Dockerfile: FROM, RUN, COPY, CMD, EXPOSE — layer caching
- Multi-stage builds: compile C++ in builder, copy binary to slim runtime
- docker-compose.yml: services, networks, volumes, healthcheck
- Nginx: reverse proxy, SSL (Let's Encrypt), gzip, rate limiting
- Prometheus + Grafana: metrics and dashboards
- Structured logging (JSON format) — grep-friendly, ELK-ingestible

Build Tasks:

- ◆ Dockerise crawler, API, frontend
- ◆ docker compose up — entire stack in one command
- ◆ Nginx: proxy to FastAPI + static files + gzip + rate limiting
- ◆ Deploy to Oracle Cloud Free Tier VM
- ◆ System crawls 100 pages on startup, search returns results in <200ms
- ◆ 4 crawler replicas — Kafka receives messages from all replicas
- ◆ UptimeRobot monitoring — ping every 5 min
- ◆ Prometheus+Grafana: expose /metrics from FastAPI — dashboard P95 latency
- Week 10 Output: Full system deployed — Nginx+FastAPI+Postgres+Redis+RocksDB in Docker, live at public URL.

★ WEEK 11 — Deep Testing: Unit, Integration, Stress

Days 61–67. This week fills the biggest gap in v2. Build a comprehensive test suite that gives you confidence before Week 12 load testing.

Day 61 — C++ Testing with GoogleTest

Core Concepts:

- GoogleTest (gtest): TEST(), TEST_F(), EXPECT_*/ASSERT_* macros
- Test fixtures: SetUp()/TearDown() for shared state
- Parameterised tests: TEST_P() for table-driven testing
- Mock objects: gmock — MOCK_METHOD, EXPECT_CALL
- Coverage: gcov + lcov — target ≥80% line coverage for core modules
- CTest integration with CMake — run all tests with 'make test'

Build Tasks:

- ◆ Install GoogleTest via CMake FetchContent
- ◆ Write tests for DNSResolver: cache hit, cache miss, TTL expiry, DNS failure
- ◆ Write tests for HTTPFetcher: 200 OK, 301 redirect, 404, timeout, partial recv
- ◆ Write tests for LRU Cache: eviction order, capacity limit, thread safety
- ◆ Write tests for Bloom Filter: insert, query, FPR at 100K inserts, serialise/deserialise
- ◆ Write tests for URL normaliser: 20 edge cases (relative paths, encoding, tracking params)
- ◆ Run gcov — reach 80% line coverage on all crawler modules
- ◆ Add 'cmake --build . --target test' to your Makefile

★ New / Extended Tasks:

- ★ Mock the network in HTTPFetcher tests using a fake socket server (localhost echo server) — no real internet needed
- ★ Test thread pool under SIGINT: submit 1000 tasks, send interrupt, verify graceful shutdown — no tasks dropped
- ★ Fuzz URL parser with libFuzzer: generate 10K random URL strings — zero crashes

Day 62 — Python Testing with pytest

Core Concepts:

- pytest: test discovery, fixtures, parametrize, marks
- unittest.mock: Mock, MagicMock, patch — mock external I/O
- pytest-cov: coverage reports — target ≥85% for pipeline code
- pytest-asyncio: testing async FastAPI endpoints
- httpx.AsyncClient: in-process FastAPI testing without running a server
- Factory fixtures: generate test documents, fake HTML pages

Build Tasks:

- ◆ Tests for html_parser: boilerplate removal, Unicode normalisation, encoding detection
- ◆ Tests for lang_detector: English, Hindi, Tamil — mock fastText model
- ◆ Tests for tokenizer: English Snowball, Stanza dispatch, indic_tokenize fallback
- ◆ Tests for NER: known entity strings, empty input, truncated input
- ◆ Tests for BM25: known query/doc pairs with hand-computed expected scores
- ◆ Tests for PageRank: 3-node graph with known analytical solution
- ◆ FastAPI endpoint tests: /search, /autocomplete, /api/click — use httpx.AsyncClient
- ◆ Run pytest --cov — reach 85% line coverage on pipeline + backend

★ New / Extended Tasks:

- ★ Property-based tests with Hypothesis: generate random token lists → verify BM25 score is always ≥0
- ★ Contract test: NLP pipeline output schema validated by Pydantic — every field present and correct type
- ★ Snapshot tests: 10 hand-curated queries → assert result order matches saved golden file

Day 63 — Integration Testing

Core Concepts:

- Integration tests: test two or more real components working together
- Docker Compose test environment: spin up real Postgres, Redis, RocksDB
- pytest fixtures with scope='session' — one DB per test run
- Test data management: seed DB before tests, rollback/truncate after
- End-to-end smoke test: crawl 10 pages → NLP pipeline → index → query → assert non-empty results
- Idempotency test: run same crawl twice — no duplicate documents in DB

Build Tasks:

- ◆ Integration test: insert 100 docs into RocksDB via index_builder → query each term → assert correct postings
- ◆ Integration test: push 10 messages to Kafka 'raw_pages' → consumer processes all → assert DB contains 10 rows
- ◆ Integration test: /search endpoint hits real Redis cache — first call slow, second call fast (<5ms)
- ◆ Integration test: full pipeline — crawl 10 URLs → NLP → index → search for a known term → assert rank 1 is correct URL
- ◆ Idempotency: crawl same URL twice → assert only one DB row, content_hash unchanged

★ New / Extended Tasks:

- ★ Write a test that kills the Kafka broker mid-pipeline — assert crawler retries and no data loss
- ★ Write a test that fills Redis to memory limit — assert graceful degradation (cache miss, not crash)
- ★ Database migration test: apply schema migrations, verify indexes exist, run EXPLAIN on key queries

Day 64 — Stress Testing the Crawler

Core Concepts:

- Stress test vs load test: stress = find breaking point; load = verify normal throughput
- Metrics during stress: CPU%, memory, open file descriptors, thread count
- Crawler stress targets: ≥50 pages/sec with 64 threads on 100Mbps link
- Memory leak detection under load: valgrind --tool=massif
- File descriptor leak: lsof | grep your_binary | wc -l — should plateau
- Connection pool exhaustion: too many sockets in TIME_WAIT state

Build Tasks:

- ◆ Stress test: crawl 10,000 pages with 64 threads — measure pages/sec, MB/sec, error rate
- ◆ Monitor /proc/{pid}/status during crawl — RSS should not grow unboundedly
- ◆ Run valgrind --tool=massif on 1,000-page crawl — inspect heap profile with ms_print
- ◆ Monitor open file descriptors with 'watch -n1 lsof | grep crawler | wc -l' — confirm plateau
- ◆ Intentionally exhaust thread pool queue (submit 1M tasks) — measure back-pressure behaviour

★ New / Extended Tasks:

- ★ Chaos: inject 50% HTTP timeout rate via a proxy — assert crawler retries correctly and throughput degrades gracefully
- ★ DNS failure injection: override /etc/hosts to fail 20% of domains — verify exponential backoff, no crash
- ★ Long-running stability test: crawl for 2 hours continuously — assert no memory growth (RSS stable ±5%)

Day 65 — Stress Testing the Search Pipeline

Core Concepts:

- NLP pipeline throughput target: ≥200 documents/sec per worker
- Kafka consumer lag monitoring: consumer group lag should not grow unboundedly
- RocksDB write throughput: ≥100K postings/sec during index build
- Index build time: 1M documents should index in <30 minutes
- Query latency under load: P50, P95, P99 — all at realistic concurrent load

Build Tasks:

- ◆ Benchmark NLP pipeline: measure docs/sec for 10K document batch
 - ◆ Index 100K documents — measure total time and RocksDB write throughput
 - ◆ Run 8 Kafka consumer workers — measure aggregate throughput, confirm linear scaling
 - ◆ Monitor Kafka consumer lag with kafka-consumer-groups.sh during bulk ingest
 - ◆ BM25 query latency: 1000 queries against 100K document index — measure P50/P95/P99
-

Day 66 — API Load Testing with Locust + k6

Core Concepts:

- Locust: Python-based load testing framework — define user behaviour as code
- k6: Go-based, VU (virtual user) model, JavaScript scenarios
- wrk2: constant-throughput HTTP benchmarking — accurate latency percentiles
- SLA targets for search API: P95 < 200ms at 100 concurrent users
- Error rate target: <0.1% 5xx errors at nominal load
- Ramp-up testing: slowly increase load until P95 exceeds 500ms (your capacity ceiling)

Build Tasks:

- ◆ Locust scenario: search user (70% search, 20% autocomplete, 10% click) — 100 users, 5min
- ◆ k6 scenario: ramp from 10 to 200 VUs over 5 minutes — capture P95 at each step
- ◆ wrk2 steady-state: 100 req/s for 60s — measure P50/P95/P99/P999 accurately
- ◆ Identify and fix the first bottleneck found (often Redis connection pool or Postgres query plan)
- ◆ Re-run after fix — verify P95 improved by ≥30%

★ New / Extended Tasks:

- ★ Spike test: jump from 10 to 500 users in 5 seconds — assert system recovers within 30s (no OOM, no crash)
 - ★ Soak test: 50 concurrent users for 8 hours — assert no memory leak, no latency degradation over time
 - ★ Baseline report: capture and commit a JSON file with P50/P95/P99 for future regression detection
-

Day 67 — Week 11 Testing Checkpoint

Core Concepts:

- Test pyramid: unit (fast, many) → integration (medium) → e2e (slow, few)
- Coverage is not quality: 80% covered but wrong assertions = false confidence
- Regression baseline: save benchmark numbers now — compare after every major change

Build Tasks:

- ◆ Run full test suite (gtest + pytest): all tests green, zero warnings
- ◆ Coverage report: C++ ≥80%, Python ≥85%
- ◆ Benchmark baseline: crawler pages/sec, pipeline docs/sec, query P95 — commit to BENCHMARKS.md
- ◆ Write a test that will catch any future performance regression >20% on BM25 query latency
- Week 11 Output: Comprehensive test suite (unit+integration+stress), coverage ≥80%, benchmark baseline committed.

★ WEEK 12 — Observability, Profiling & Advanced Load Testing

Days 68–74. You can't improve what you can't measure. Instrument everything.

Days 68–69 — Metrics with Prometheus + Grafana

Core Concepts:

- Four golden signals: latency, traffic, errors, saturation
- Prometheus data model: metric name + labels + timestamp + value
- Metric types: Counter, Gauge, Histogram, Summary
- prometheus-fastapi-instrumentator: auto-instrument FastAPI
- Custom metrics: `index_build_duration_seconds`, `crawl_pages_total`, `query_bm25_score_histogram`
- Grafana: connect to Prometheus, build dashboards, set alert thresholds

Build Tasks:

- ◆ Install Prometheus + Grafana via Docker Compose
- ◆ Add `/metrics` to FastAPI using `prometheus-fastapi-instrumentator`
- ◆ Define custom metrics: `search_query_latency_seconds` (Histogram), `search_results_count` (Gauge), `crawler_pages_total` (Counter)
- ◆ Build Grafana dashboard: request rate, P50/P95/P99 latency, error rate, crawler throughput
- ◆ Set alert rule: PagerDuty/email if P95 > 500ms for 5 consecutive minutes
- ◆ Add metrics to C++ crawler via Prometheus C++ client or expose to Redis + scrape from Python

★ New / Extended Tasks:

- ★ Track query-specific metrics: zero-result rate, avg result count, cache hit rate — add to Grafana
 - ★ Add SLO dashboard: 99.9% of queries served in <200ms — visualise error budget burn rate
 - ★ ClickHouse query analytics: top-100 most searched terms, CTR by rank position, dwell time histogram
-

Days 70–71 — Structured Logging + Distributed Tracing

Core Concepts:

- Structured logging: JSON logs with `request_id`, `user_id`, `latency_ms`, `status`
- Log levels: DEBUG/INFO/WARNING/ERROR/CRITICAL — use appropriately
- Correlation IDs: propagate `request_id` through crawler → pipeline → index → query
- OpenTelemetry: vendor-neutral tracing standard
- Jaeger: distributed trace visualisation — see latency breakdown per span
- ELK Stack: Elasticsearch + Logstash + Kibana for log aggregation

Build Tasks:

- ◆ Replace all `print()` with `structlog` or Python logging — JSON format with `request_id`
- ◆ Add correlation ID header (X-Request-ID) to FastAPI — propagate to Redis + Postgres calls
- ◆ Instrument FastAPI with OpenTelemetry SDK — export traces to Jaeger
- ◆ Add trace spans for: BM25 scoring, PageRank lookup, Redis cache check, Postgres query
- ◆ Run a slow query — find it in Jaeger trace — identify which span is the bottleneck

★ New / Extended Tasks:

- ★ Add trace context to Kafka messages — trace a document from crawl to query result end-to-end
 - ★ Log all zero-result queries with the query text — analyse weekly to find index gaps
 - ★ Set up log-based alerting: alert if ERROR log rate exceeds 1% of request volume
-

Days 72–73 — C++ Profiling with perf + Flamegraphs

Core Concepts:

- perf: Linux performance counters — CPU cycles, cache misses, branch mispredictions
- perf stat: aggregate statistics for a run
- perf record + perf report: sample-based profiling
- Flamegraphs (Brendan Gregg): visualise CPU time by call stack
- Valgrind callgrind: instruction-level call graph profiling
- gprof: GNU profiler — compile with -pg
- Hotspot identification: where does the crawler spend >10% of CPU time?

Build Tasks:

- ◆ perf stat on crawler — report L1/L3 cache miss rate during active crawl
- ◆ perf record -g ./crawler | perf script | FlameGraph/stackcollapse-perf.pl | flamegraph.pl > out.svg
- ◆ Open flamegraph in browser — identify the hottest function
- ◆ Valgrind callgrind on index_builder — identify most-called function
- ◆ Optimise the identified hotspot — re-measure — document speedup in BENCHMARKS.md

★ New / Extended Tasks:

- ★ Profile BM25 query scoring on 100K document index — use callgrind to count instructions per query
- ★ Profile RocksDB read path: measure time in decode_postings vs RocksDB::Get vs network deserialisation
- ★ Identify and eliminate one false-sharing cache line in the thread pool using perf c2c

Day 74 — Python Profiling with py-spy + cProfile

Core Concepts:

- cProfile: built-in Python profiler — deterministic, overhead ~2x
- py-spy: sampling profiler — attach to running process, zero overhead on production
- line_profiler: line-by-line hotspot identification
- memory_profiler: track memory allocations per line
- Asyncio profiling: aiomonitor, asyncio debug mode
- Common Python hotspots: JSON serialisation, regex compilation, Stanza model loading

Build Tasks:

- ◆ python -m cProfile -o pipeline.prof consumer.py — analyse with pstats or snakeviz
- ◆ py-spy record -o profile.svg -- python consumer.py — open flamegraph
- ◆ Identify the top-3 hotspots in the NLP pipeline — document them
- ◆ Profile FastAPI under load: py-spy top --pid \$(pgrep uvicorn) — observe during Locust run
- ◆ Fix one hotspot (e.g. preload Stanza model once not per-document) — measure speedup
- Week 12 Output: Full observability stack (Prometheus+Grafana+Jaeger) + flamegraphs for C++ and Python + profiling report.

★ WEEK 13 — Advanced IR: LTR, Vector Search, Neural Ranking

Days 75–81. Go beyond BM25. These techniques separate a toy search engine from a production one.

Days 75–76 — Learning-to-Rank (LTR)

Core Concepts:

- Why LTR: BM25+PageRank is a heuristic; LTR learns the optimal blend from data
- Feature engineering: BM25 score, PageRank, URL length, title match, dwell time, CTR
- Pointwise LTR: predict relevance score per (query, doc) — regression
- Pairwise LTR: prefer doc_i over doc_j — RankNet, LambdaRank
- Listwise LTR: optimise nDCG directly — LambdaMART (XGBoost/LightGBM)
- Training data: click logs → implied relevance (clicks as positive, skips as negative)
- Position bias correction: inversely weight clicks by rank position

Build Tasks:

- ◆ Collect 1000 (query, doc, click) triples from query_log table
- ◆ Build feature matrix: [bm25_score, pagerank, title_match, url_length, lang_match, dwell_ms]
- ◆ Train LambdaMART model using LightGBM with rank objective
- ◆ Evaluate offline: nDCG@10 with LTR vs BM25+PR on held-out 200 queries
- ◆ Integrate into QueryEngine.rerank(): score = ltr_model.predict(features)

★ New / Extended Tasks:

- ★ Feature importance analysis: which feature contributes most to ranking quality?
 - ★ A/B test: BM25+PR vs LTR — collect 1 week of clicks — compare nDCG@10
 - ★ Online learning: retrain LTR model weekly from accumulated click logs via scheduled job
-

Days 77–78 — Dense Vector Search + FAISS

Core Concepts:

- Dense retrieval: encode query and docs as high-dimensional vectors, retrieve by cosine/dot similarity
- Sentence embeddings: sentence-transformers (SBERT), IndicBERT for Indian languages
- FAISS: Facebook AI Similarity Search — exact (Flat) and approximate (IVF, HNSW) indexes
- HNSW: Hierarchical Navigable Small World — fast ANN with tunable recall/speed tradeoff
- Hybrid retrieval: BM25 score + dense score → rerank (reciprocal rank fusion)
- Re-ranking with cross-encoder: slow but more accurate — use as final-stage reranker

Build Tasks:

- ◆ Encode 10K documents with sentence-transformers/paraphrase-MiniLM-L6-v2
- ◆ Build FAISS IVFFlat index — query with 100 test queries — measure recall@10
- ◆ Build FAISS HNSW index — compare QPS vs IVFFlat at same recall@10
- ◆ Hybrid search: merge BM25 top-50 + FAISS top-50 → rerank with reciprocal rank fusion
- ◆ Benchmark: dense-only vs BM25-only vs hybrid on your 200-query benchmark — compare nDCG@10

★ New / Extended Tasks:

- ★ Encode Hindi documents with AI4Bharat IndicBERT — add to separate FAISS index
 - ★ Cross-encoder reranking: use cross-encoder/ms-marco-MiniLM-L-6-v2 on top-20 candidates
 - ★ ANN recall-QPS tradeoff curve: plot recall@10 vs QPS for ef_search=16,32,64,128,256 in HNSW
-

Days 79–80 — Query Understanding + Expansion

Core Concepts:

- Query classification: navigational vs informational vs transactional

- Named entity recognition on queries: 'Modi speech' → entity:PERSON → boost entity docs
- Query expansion: synonym expansion (WordNet), spelling correction (SymSpell), query reformulation
- Pseudo-relevance feedback (PRF): expand query with terms from top-k retrieved docs
- Cross-lingual query: detect Roman-script Hindi → transliterate → fan-out to Hindi index
- Query segmentation: 'new york times' → phrase → positional index lookup

Build Tasks:

- ◆ Integrate SymSpell for spelling correction — test on 100 misspelled queries
- ◆ WordNet synonym expansion for English queries — measure recall change
- ◆ PRF: expand query with top-5 terms from top-10 retrieved docs — measure nDCG change
- ◆ Entity-aware reranking: if query contains PERSON entity, boost pages with matching entity in DB
- ◆ Query intent classifier: logistic regression on query features → navigational/informational/transactional

Day 81 — Week 13 Evaluation Checkpoint

Core Concepts:

- Full offline evaluation pipeline: run all ranking variants on 200-query benchmark
- Statistical significance: paired t-test or Wilcoxon signed-rank to compare variants
- Ablation study: measure contribution of each component independently

Build Tasks:

- ◆ Run all 4 ranking variants on 200-query benchmark: TF-IDF, BM25+PR, LTR, Hybrid (BM25+Dense)
- ◆ Build comparison table: P@5, P@10, R@10, MAP, nDCG@10 for each variant
- ◆ Statistical test: is LTR significantly better than BM25+PR? ($p < 0.05$?)
- ◆ Ablation: remove each LTR feature one-at-a-time — identify most impactful feature
- ◆ Commit results to EVALUATION.md — your permanent record of ranking quality
- Week 13 Output: LTR model trained, FAISS vector index deployed, hybrid search live, full evaluation report.

★ WEEK 14 — Production Hardening, CI/CD & Capstone Review

Days 82–90. Harden the system, automate deployment, inject faults, and do a final complete review.

Days 82–83 — Security Hardening

Core Concepts:

- Input sanitisation: strip HTML from user queries before passing to search engine
- SQL injection prevention: parameterised queries everywhere — no string concatenation
- Rate limiting: slowapi per-IP + global limits on /search and /crawl endpoints
- robots.txt edge cases: malformed file, 5xx response, 403 (treat as no restrictions)
- SSRF prevention: validate crawl targets against allowlist — prevent crawling internal IPs
- Secrets management: environment variables only — never hardcode credentials
- Dependency audit: pip-audit, safety, npm audit — fix critical CVEs

Build Tasks:

- ◆ Fuzz /search endpoint with sqlmap (read-only scan) — assert no injection vectors
 - ◆ Test rate limiter: 200 requests in 1 second — assert 429 after limit hit
 - ◆ Test SSRF: attempt to crawl 127.0.0.1, 169.254.169.254 — assert blocked
 - ◆ Run pip-audit on requirements.txt — fix all HIGH/CRITICAL vulnerabilities
 - ◆ Add Content-Security-Policy, X-Frame-Options, X-Content-Type-Options headers to Nginx
 - ◆ Verify all Postgres queries use parameterised form — grep codebase for string format in SQL
-

Days 84–85 — CI/CD Pipeline with GitHub Actions

Core Concepts:

- CI: run tests automatically on every push and pull request
- CD: deploy automatically when main branch tests pass
- GitHub Actions: .github/workflows/ci.yml
- Pipeline stages: lint → build → unit test → integration test → coverage → deploy
- Docker layer caching in CI: cache FROM layers to speed builds
- Branch protection: require CI pass before merge to main
- Deployment environments: staging (auto-deploy) → production (manual approval)

Build Tasks:

- ◆ Create .github/workflows/ci.yml: lint (clang-format, black), build C++, run gtest, run pytest
 - ◆ Add integration test job: spin up docker compose, run integration tests, tear down
 - ◆ Add coverage gate: fail if C++ coverage <80% or Python <85%
 - ◆ Add performance regression gate: run wrk2 on staging, fail if P95 regresses >20%
 - ◆ Create .github/workflows/deploy.yml: build Docker images, push to registry, SSH deploy to Oracle Cloud
 - ◆ Set up branch protection on main: require CI pass, require 1 approval
-

Days 86–87 — Chaos Engineering + Fault Injection

Core Concepts:

- Chaos engineering: deliberately inject failures to test system resilience
- Failure modes: container crash, network partition, disk full, CPU spike, memory pressure
- Steady-state hypothesis: define 'normal' before injecting chaos
- Retry + backoff: every external call should have idempotent retry logic
- Circuit breaker pattern: stop calling a failing service after N consecutive failures
- Graceful degradation: if Redis is down, serve requests without cache (slower but functional)

Build Tasks:

- ♦ Kill Postgres container during search load — assert API returns 503 (not 500), recovers within 30s after restart
- ♦ Kill Redis container — assert search still works (no cache) with latency increase only
- ♦ Kill Kafka broker — assert crawler pauses and resumes without data loss after restart
- ♦ Network partition: use tc netem to add 500ms latency to Postgres — assert P95 degrades gracefully
- ♦ Fill disk to 95%: assert structured log appears, crawler pauses, no silent data loss
- ♦ Memory pressure: set container memory limit to 256MB on backend — assert OOM kill triggers restart, not silent failure

Days 88–89 — Final Performance Tuning

Core Concepts:

- Connection pooling tuning: optimal pool size = (core_count * 2) + effective_spindle_count
- Postgres: tune work_mem, shared_buffers, effective_cache_size, max_connections
- RocksDB: tune block_cache_size, write_buffer_size, max_open_files
- Redis: tune maxmemory, maxmemory-policy (allkeys-lru for cache)
- Nginx: tune worker_processes, worker_connections, keepalive_timeout
- Async query fan-out: use asyncio.gather() for parallel shard queries

Build Tasks:

- ♦ Profile query path under 100 concurrent users — identify top-3 bottlenecks with Jaeger
- ♦ Tune Postgres: add connection pool (pgBouncer or SQLAlchemy pool=20) — measure P95 improvement
- ♦ Tune RocksDB block_cache to 1GB — measure query latency improvement on repeated terms
- ♦ Replace sequential shard fan-out with asyncio.gather() — measure latency reduction
- ♦ Final load test: 100 concurrent users, 5 minutes — target: P95 <150ms, error rate <0.01%
- ♦ Commit final BENCHMARKS.md: before/after for every optimisation performed

Day 90 — Capstone Review + Final Checklist

The final day is a structured self-audit. Review every component against the rubric below.

Component	Minimum Viable	Production Grade	Pass?
C++ Crawler	Fetches HTML, saves to disk, single-threaded	64-thread pool, robots.txt, rate limiting, Bloom dedup, Kafka output, 50+ pages/sec	
NLP Pipeline	Tokenises English text	Lang detection, stemming/lemma for 10 Indian langs, NER, Kafka I/O, 200+ docs/sec	
Inverted Index	In-memory unordered_map	SPIMI on disk, delta+VByte compression, 8-shard RocksDB, zstd, field weights	
BM25 Ranking	TF-IDF scoring	BM25 K1=1.5 B=0.75, PageRank fusion 0.7/0.3, LTR reranking, hybrid dense retrieval	
Query Engine	Returns matching docs	<150ms P95 at 100 concurrent users, snippet gen, spell correction, cross-lingual	
FastAPI Backend	GET /search returns JSON	/search + /autocomplete + /api/click, Redis cache, rate limiting, OpenAPI docs	
Frontend	HTML form + result list	Snippets, highlighting, pagination, language selector, entity tags, click+dwel tracking	
Deployment	Runs locally	Docker Compose, Nginx SSL, Oracle Cloud, UptimeRobot, auto-restart on failure	
Testing	No tests	Unit ≥80%, Integration ≥key flows, Stress (50 pages/sec), Load (P95<200ms@100VUs)	
Observability	print() debugging	Prometheus+Grafana dashboards, Jaeger traces, structured JSON logs, alerting rules	

Component	Minimum Viable	Production Grade	Pass?
Evaluation	Subjective assessment	200-query benchmark, nDCG@10, MAP, A/B test with Mann-Whitney U, CTR/dwell online	
CI/CD	Manual deployment	GitHub Actions: lint+test+coverage gates + auto-deploy to staging on merge to main	

■ Day 90: If every row in the table above is marked Pass, you have built a production-grade distributed search engine. Congratulations.

Appendix A — System Architecture Quick Reference

A.1 — End-to-End Data Flow

Stage	Component	Technology	Output
1. Seed	URL Frontier	Redis Priority Queue + Bloom Filter	Deduplicated URL stream
2. Fetch	C++ Crawler (64 threads)	Raw sockets, DNS LRU, OpenSSL	Raw HTML → MinIO (SHA256 key)
3. Transport	Message Queue	Apache Kafka 'raw_pages' topic	HTML messages to pipeline
4. Parse	HTML Parser	lxml, BeautifulSoup	title, body, h1/h2, links[]
5. Detect	Language Detector	fastText lid.176.bin (176 langs)	ISO 639-1 lang code
6. NLP	Tokeniser + Stemmer	NLTK Snowball (EN), Stanza (10 Indian langs)	token list, lemmas
7. NER	Entity Extractor	spaCy (EN), Stanza NER (Indian)	[(text, label)] list
8. Index	Index Builder (SPIMI)	Python → RocksDB (8 shards, zstd)	Posting lists on disk
9. Meta data	Document Store	PostgreSQL (urls, links, entities, query_log)	Relational metadata
10. Rank	Query Engine	BM25 + PageRank + LTR + Dense FAISS	Top-k scored results
11. Serve	FastAPI Backend	uvicorn, Redis cache (TTL=300s), asyncio	JSON search results
12. Display	Jinja2 Frontend	HTML templates, HTMX, click beacon JS	Search results page
13. Proxy	Nginx	SSL termination, gzip, rate limiting	Public HTTPS endpoint

A.2 — Storage Choice Summary

Data	Store	Why	CAP
Document metadata	PostgreSQL	ACID, relational joins, FTS via tsvector	CP
Inverted posting lists	RocksDB (LSM)	Write-optimised, compressed SSTs, fast range scan	AP
URL frontier + seen-set	Redis + RedisBloom	In-memory BLPOP, Bloom Filter for dedup	CP
PageRank scores	PostgreSQL (urls.pagerank)	JOIN with metadata for reranking	CP
Raw HTML blobs	MinIO (S3-compatible)	Content-addressed, cheap, dedup by SHA256	AP
Query/result cache	Redis (TTL=300s)	Sub-millisecond cache hit, TTL eviction	AP
Analytics + A/B logs	ClickHouse	Columnar OLAP, CTR by rank, dwell histogram	AP
Dense vectors	FAISS (HNSW)	ANN search, recall-speed tradeoff	AP

Appendix B — Complete Testing & Benchmarking Reference

B.1 — Test Categories and Tools

Category	Language	Tool	What to Test	Coverage Target
Unit	C++	GoogleTest + gmock	DNS, HTTP, LRU, Bloom, URL parser, thread pool	≥80% line
Unit	Python	pytest + unittest.mock	HTML parser, tokeniser, BM25, PageRank, NER, LTR	≥85% line
Property-based	Python	Hypothesis	BM25 score ≥0, nDCG in [0,1], tokeniser idempotent	N/A
Contract	Python	Pydantic + pytest	Pipeline output schema validation end-to-end	100% schemas
Integration	Python	pytest + Docker Compose	Crawler→Kafka→Pipeline→DB→Query full flow	Key flows
Snapshot/Golden	Python	pytest	10 curated queries → assert result order stable	10 queries
Stress (Crawler)	C++/bash	custom + perf	50+ pages/sec at 64 threads, stable RSS, 0 leaks	2hr soak
Stress (Pipeline)	Python	custom	200+ docs/sec per worker, Kafka lag stays bounded	1hr soak
Load (API)	Python/Go	Locust, k6, wrk2	P95 <200ms at 100 VUs, error <0.1%	5min+
Spike	Python	Locust	10→500 VUs in 5s — recover within 30s	One-off
Soak	Python	Locust	50 VUs for 8hr — no memory growth, no latency drift	Weekly
Chaos	bash	Docker, tc netem	Kill Postgres/Redis/Kafka, network delay, disk full	Per release
Security	bash/Python	sqlmap (read-only), pip-audit	SQLi, SSRF, CVEs in deps, header checks	Pre-deploy
Fuzz	C++	libFuzzer	URL parser, HTML parser — zero crashes on random input	Before v1.0

B.2 — Profiling Toolchain

Tool	Language	Type	How to Use	Output
perf stat	C++	CPU counters	perf stat ./crawler	Cache miss rate, IPC, branches
perf record + FlameGraph	C++	Sampling	perf record -g -F 99 ./crawler	SVG flamegraph
Valgrind callgrind	C++	Instr count	valgrind --tool=callgrind ./crawler	Function call graph
Valgrind massif	C++	Heap profiling	valgrind --tool=massif ./crawler	Heap growth over time

Tool	Language	Type	How to Use	Output
AddressSanitizer	C++	Memory errors	compile -fsanitize=address	Buffer overflow reports
ThreadSanitizer	C++	Race detection	compile -fsanitize=thread	Data race reports
cProfile	Python	Deterministic	python -m cProfile -o out.prof script.py	Function call stats
py-spy	Python	Sampling	py-spy record -o profile.svg -- python script.py	SVG flamegraph
line_profiler	Python	Line-level	@profile decorator + kernprof	Time per line
memory_profiler	Python	Memory	@profile decorator + python -m memory_profiler	Memory per line

Appendix C — Indian Language NLP Reference

C.1 — Supported Languages

Language	Script	ISO	Tokenizer	Stemmer/Lemma
Hindi	Devanagari	hi	Stanza / IndicNLP	Stanza lemma
Tamil	Tamil	ta	Stanza	Stanza lemma
Telugu	Telugu	te	Stanza	Stanza lemma
Malayalam	Malayalam	ml	Stanza	Stanza lemma
Bengali	Bengali	bn	Stanza	Stanza lemma
Kannada	Kannada	kn	IndicNLP	Rule-based
Gujarati	Gujarati	gu	IndicNLP	Rule-based
Marathi	Devanagari	mr	Stanza	Stanza lemma
Punjabi	Gurmukhi	pa	IndicNLP	Rule-based
Urdu	Nastaliq	ur	IndicNLP	Rule-based

C.2 — Key Libraries

Library	Purpose	Install	Notes
Stanza (Stanford NLP)	Tokenize, lemma, NER for 70+ langs	pip install stanza	Download models per language
IndicNLP Library	Tokenize + normalize Indian scripts	pip install indic-nlp-library	Covers 12 Indian languages
AI4Bharat IndicBERT	Multilingual BERT for Indian langs	pip install transformers	12 Indian languages, dense retrieval
indictrans	Script transliteration (Roman↔Devanagari)	pip install indic-transliteration	Roman-script Hindi query handling
fastText lid.176.bin	Language identification — 176 langs	Download from fasttext.cc	confidence > 0.7 threshold
NLTK Snowball	English stemming	pip install nltk	SnowballStemmer('english')

Appendix D — Evaluation Metrics Cheat Sheet

D.1 — Offline Metrics

Metric	Formula	Range	Primary Use	Target
P@k	$ \text{relevant} \cap \text{top-}k / k$	[0,1]	Precision at rank k	P@10 > 0.6
R@k	$ \text{relevant} \cap \text{top-}k / \text{relevant} $	[0,1]	Recall at rank k	R@10 > 0.5
MAP	Mean of AP over all queries	[0,1]	Ranked retrieval quality	> 0.4
nDCG@10	DCG@10 / IDCG@10	[0,1]	Graded relevance (PRIMARY)	> 0.6
MRR	Mean of 1/rank of first hit	[0,1]	First-result quality	> 0.5
F1@k	$2 * P@k * R@k / (P@k + R@k)$	[0,1]	Harmonic mean	> 0.5

D.2 — Online Metrics

Metric	Definition	Collection	Target
CTR@k	Clicks / Impressions at rank k	JS beacon on click	> 30% CTR@1
Dwell Time	Time between click and back-button	JS unload event	> 30s median
Zero-Result Rate	Queries with 0 results / total	query_log	< 5%
Abandonment Rate	Queries with no click at all	session analysis, ClickHouse	< 40%
Query Latency P95	95th percentile response time	Prometheus Histogram	< 200ms
Long-Click Rate	Dwell > 30s / total clicks	ClickHouse aggregation	> 50%
Index Freshness	Avg age of top-10 results	crawled_at from Postgres	< 7 days

D.3 — A/B Testing Protocol

Control (A): BM25+PageRank. Treatment (B): BM25+PageRank+LTR. Use stable SHA-256 hash of '{experiment}:{session_id}' for variant assignment — same session always in same variant. Run after 1 week post-deploy, minimum 10K queries per variant. Statistical test: Mann-Whitney U (non-parametric). Minimum detectable nDCG improvement: 0.5%. Confidence level: 95%.

Appendix E — Performance Targets & SLA Reference

E.1 — Component SLAs

Component	Metric	Target (Student)	Target (Production)	Measurement
C++ Crawler	Pages/sec (64 threads)	≥ 20 pages/sec	≥ 200 pages/sec	Crawler stats log
C++ Crawler	Memory (RSS) stability	< 500MB after 1hr	< 2GB after 24hr	Valgrind massif
NLP Pipeline	Documents/sec per worker	≥ 50 docs/sec	≥ 500 docs/sec	pipeline metrics
Index Build	1M documents total time	< 2 hours	< 30 minutes	wall clock
BM25 Query (10K docs)	P50 latency	< 10ms	< 1ms	wrk2
BM25 Query (1M docs)	P95 latency	< 100ms	< 20ms	wrk2
FastAPI /search (50 VUs)	P95 latency	< 200ms	< 100ms	Locust/k6
FastAPI /search (100 VUs)	P95 latency	< 500ms	< 200ms	Locust/k6
FastAPI /search	Error rate	< 0.1% (5xx)	< 0.01% (5xx)	Prometheus
Redis cache hit	Latency	< 5ms	< 1ms	Prometheus histogram
Postgres metadata query	P95 latency	< 50ms	< 10ms	EXPLAIN ANALYZE
RocksDB posting list read	P95 latency	< 10ms	< 2ms	custom benchmark
Full search (end-to-end)	P95 at public URL	< 500ms	< 200ms	UptimeRobot
System availability	Uptime	≥ 95%	≥ 99.9%	UptimeRobot

E.2 — How to Measure Each Target

Measurement	Tool	Command / How
API P50/P95/P99	wrk2	wrk2 -t4 -c100 -d60s -R100 --latency http://localhost:8000/search?q=india
API sustained load	Locust	locust -f locustfile.py --headless -u 100 -r 10 --run-time 5m
API gradual ramp	k6	k6 run --vus 10 --stage 5m:200 script.js
Crawler throughput	Custom	Count pages/sec from log: grep 'fetched' crawler.log pv -l > /dev/null
NLP pipeline throughput	Custom	time python consumer.py --batch 10000 2>&1 grep real
RocksDB latency	Custom	Python benchmark: 10K random term lookups, measure with time.perf_counter()
Postgres query time	psql	EXPLAIN (ANALYZE, BUFFERS) SELECT ... — look at 'Execution Time'
Memory leak detection	Valgrind massif	valgrind --tool=massif --pages-as-heap=yes ./crawler
CPU flamegraph	perf + FlameGraph	perf record -g -F 99 -p PID -- sleep 30 && perf script stackcollapse-perf.pl flamegraph.pl > cpu.svg
Full trace (distributed)	Jaeger	Open http://localhost:16686 → find trace by request_id → inspect spans

The Only Rule That Matters

Build it. Ship it. Test it. Measure it. Break it. Fix it. Learn for life. Every concept in this plan has a build task attached. Every build task has a test attached. Every test has a performance target attached. If you finished reading about BM25 without implementing it from scratch, you haven't learned BM25. If you implemented BM25 without measuring its P95 query latency, you don't know if it's production-ready. The search engine you build in 14 weeks will teach you more systems engineering than 3 years of courses alone.